

Navigation, Planning, and Safety-Critical Control for Small Autonomous Marine Vehicles: A Literature Review

Prajjwal Dutta

Email: pdutta9@asu.edu

Abstract—Autonomous underwater vehicles (AUVs) and other small marine robots increasingly operate in complex, cluttered, and communication-limited environments. Reliable operation in these regimes requires the tight integration of state estimation, path planning, and control under nonlinear dynamics, sensor noise, and strict safety and connectivity constraints. This paper presents a structured literature review organized around three key components of the autonomy stack for small marine vehicles: (i) nonlinear sensor fusion and state estimation (EKF, UKF, PF), (ii) classical and sampling-based path planning (A*, D*, Hybrid A*, RRT*, PRM, BIT*, FMT*, and trajectory optimization), and (iii) feedback and safety-critical control (PID, LQR, MPC, and CBF-based controllers). The review emphasizes how these components interact in the presence of safety and connectivity requirements, with a particular focus on control barrier function (CBF) methods for constraint enforcement. This synthesis provides the foundation for simulation frameworks and control architectures targeting safety-aware, connectivity-preserving multi-AUV deployments in flow-dominated, GPS-denied environments.

Index Terms—Autonomous underwater vehicles, state estimation, path planning, model predictive control, control barrier functions, connectivity maintenance.

I. INTRODUCTION

Autonomous underwater vehicles (AUVs) are increasingly used for tasks such as environmental monitoring and exploration in complex environments. These domains demand robust integration of state estimation, motion planning, and control, operating under nonlinear vehicle dynamics, noisy sensing, limited communication, and strict safety requirements. These challenges are further amplified in cluttered environments, where accurate localization, safe navigation, and coordination among multiple vehicles must be achieved using onboard sensing and relying only on local information.

This literature review is organized around three fundamental components of autonomous navigation: state estimation, path planning, and control. First, nonlinear state estimation methods commonly used in marine robotics are reviewed, with emphasis on Extended Kalman Filters (EKF), Unscented Kalman Filters (UKF), and Particle Filters (PF), which enable sensor fusion under uncertainty and nonlinearity. Second, classical and sampling-based path planning algorithms are surveyed, including grid-based, sampling-based, and optimization-based methods. Finally, control strategies ranging from classical feedback controllers (PID) to optimization-based (LQR, MPC)

and safety-critical methods (CBF) are discussed, highlighting their role in trajectory tracking and constraint enforcement.

The central idea of the review is the interaction between convergence objectives and safety constraints. We interpret the growing body of work on Control Barrier Function (CBF)-based control in light of underlying state estimators, path planning algorithms, classical controllers, and CBF-based safety constraints. The reviewed literature establishes the theoretical and practical foundation for safety-aware, connectivity-preserving autonomy, which directly motivates the simulation frameworks and control architectures developed in subsequent research.

II. SENSOR FUSION AND STATE ESTIMATION ALGORITHMS

State estimation is a fundamental requirement for autonomous marine vehicles operating in uncertain environments for localization and navigation in challenging conditions. AUVs rely on heterogeneous onboard sensors such as inertial measurement units (IMUs), depth sensors, Doppler velocity logs (DVLs), magnetometers, acoustic positioning systems, and, when available, GPS. Individually, these sensors provide incomplete and noisy measurements. Sensor fusion techniques are therefore required to estimate vehicle position, velocity, orientation, and other states in a consistent and robust manner.

This section reviews three widely used nonlinear state estimation algorithms in marine robotics: the Extended Kalman Filter (EKF), the Unscented Kalman Filter (UKF), and Particle Filters (PF). These methods differ in how they represent uncertainty, handle nonlinearities, and trade off estimation accuracy against computational complexity.

A. Extended Kalman Filter (EKF)

1) *Conceptual Description*: The Extended Kalman Filter (EKF) is the nonlinear extension of the original linear Kalman filter [1]. It assumes the system dynamics and measurement models are nonlinear but differentiable, and it locally linearizes them around the current state estimate using a first-order Taylor expansion (Jacobians).

At every iteration, the EKF performs the following steps:

- **Prediction**: Propagates the state estimate through the nonlinear system dynamics.
- **Linearization**: Computes Jacobians of the system and measurement models.

- **Update:** Applies a Kalman filter using the linearized models.

The EKF assumes Gaussian process and measurement noise. It represents uncertainty using a mean vector and covariance matrix.

2) *Mathematical Formulation:* The EKF estimates the state of a nonlinear dynamical system by propagating the mean and covariance of a Gaussian approximation to the state distribution [2].

a) *System Model:* Consider the discrete-time nonlinear system

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}, \quad \mathbf{w}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \quad (1)$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad (2)$$

where $f(\cdot)$ describes the nonlinear state transition using the previous state and control input, and $h(\cdot)$ maps the state to the measurement space. The additive noise terms model unmodeled dynamics and sensor noise.

b) *Prediction Step:* The prediction step computes the *a priori* estimate of the state before incorporating the current measurement:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}). \quad (3)$$

Since the dynamics are nonlinear, the state covariance cannot be propagated exactly. The EKF linearizes the dynamics about the current estimate using a first-order Taylor expansion. The Jacobian matrix is

$$\mathbf{F}_{k-1} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}=\mathbf{u}_{k-1}}. \quad (4)$$

The predicted covariance is

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1} \mathbf{P}_{k-1|k-1} \mathbf{F}_{k-1}^\top + \mathbf{Q}. \quad (5)$$

c) *Update Step:* The measurement model is linearized around the predicted state using the Jacobian

$$\mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{k|k-1}}. \quad (6)$$

The innovation is

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1}), \quad (7)$$

with innovation covariance

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}. \quad (8)$$

The Kalman gain is

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1}. \quad (9)$$

The posterior state estimate and covariance are updated as

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k, \quad (10)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}. \quad (11)$$

3) *Strengths and Weaknesses:*

a) *Strengths:*

- Computationally efficient, with dominant complexity from covariance updates.
- Performs well for mildly nonlinear systems with high update rates.

b) *Weaknesses:*

- Linearization errors can be significant for highly nonlinear systems.
- Requires analytic Jacobians, which can be difficult for complex models.
- Can diverge if linearization is poor or noise covariances $\mathbf{Q}$ and $\mathbf{R}$ are badly tuned.
- Assumes Gaussian noise and cannot represent multimodal uncertainty.

4) *Computational Cost:* Per time step, the EKF requires:

- One nonlinear dynamics evaluation $f(\cdot)$,
- One nonlinear measurement evaluation $h(\cdot)$,
- Two Jacobian computations,
- Covariance propagation and update with complexity $\mathcal{O}(n^3)$.

5) *Suitability for Small Autonomous Vehicles:* The EKF is well suited for small autonomous vehicles provided:

- The system dynamics are not excessively nonlinear over a single time step,
- Jacobians can be derived or approximated,
- Noise statistics are reasonably well tuned.

It is widely used for IMU–DVL–depth sensor fusion and acoustic navigation in AUVs and serves as a robust baseline estimator for higher-level planning and control layers.

6) *Open-Source Availability:*

- **ROS robot_localization:** EKF and UKF nodes for multi-sensor fusion [3].
- **FilterPy** (Python): Implements KF, EKF, UKF, and particle filters [4].
- **MATLAB/Simulink:** Built-in EKF and UKF objects for real-time state estimation [5].

B. Unscented Kalman Filter (UKF)

1) *Conceptual Description:* The Unscented Kalman Filter (UKF) addresses a key weakness of the EKF: the reliance on first-order linearization of nonlinear functions. Instead of linearizing $f(\cdot)$ and $h(\cdot)$, the UKF uses the *unscented transform* [6]. It selects a set of deterministically chosen “sigma points” that capture the mean and covariance of the state distribution, propagates each sigma point through the exact nonlinear functions, and recombines them to obtain the predicted mean and covariance.

2) *Mathematical Formulation:* The UKF considers the same discrete-time nonlinear system:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}, \quad (12)$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad (13)$$

with process and measurement noise $\mathbf{w}_{k-1}$ and $\mathbf{v}_k$.

At time step $k - 1$, the state belief is summarized by mean $\hat{\mathbf{x}}_{k-1|k-1}$ and covariance $\mathbf{P}_{k-1|k-1}$ of dimension n .

a) *Sigma Point Generation*: The UKF deterministically selects $2n + 1$ sigma points:

$$\chi_{k-1}^{(0)} = \hat{\mathbf{x}}_{k-1|k-1}, \quad (14)$$

$$\chi_{k-1}^{(i)} = \hat{\mathbf{x}}_{k-1|k-1} + [(n + \lambda)\mathbf{P}_{k-1|k-1}]_i^{1/2}, \quad i = 1, \dots, n, \quad (15)$$

$$\chi_{k-1}^{(i)} = \hat{\mathbf{x}}_{k-1|k-1} - [(n + \lambda)\mathbf{P}_{k-1|k-1}]_{i-n}^{1/2}, \quad i = n + 1, \dots, 2n. \quad (16)$$

where λ is a scaling parameter and the matrix square root is typically computed via a Cholesky decomposition. Weights $W_i^{(m)}$ and $W_i^{(c)}$ are used for computing means and covariances.

b) *Prediction Through Dynamics*: Each sigma point is propagated through the exact nonlinear dynamics:

$$\chi_{k|k-1}^{(i)} = f(\chi_{k-1}^{(i)}, \mathbf{u}_{k-1}). \quad (17)$$

The predicted state mean and covariance are

$$\hat{\mathbf{x}}_{k|k-1} = \sum_{i=0}^{2n} W_i^{(m)} \chi_{k|k-1}^{(i)}, \quad (18)$$

$$\mathbf{P}_{k|k-1} = \sum_{i=0}^{2n} W_i^{(c)} \left(\chi_{k|k-1}^{(i)} - \hat{\mathbf{x}}_{k|k-1} \right) \left(\chi_{k|k-1}^{(i)} - \hat{\mathbf{x}}_{k|k-1} \right)^\top + \mathbf{Q}. \quad (19)$$

c) *Prediction Through Measurement Model*: Sigma points are mapped into measurement space:

$$\zeta_k^{(i)} = h(\chi_{k|k-1}^{(i)}). \quad (20)$$

The predicted measurement mean and covariance are:

$$\hat{\mathbf{z}}_k = \sum_{i=0}^{2n} W_i^{(m)} \zeta_k^{(i)}, \quad (21)$$

$$\mathbf{S}_k = \sum_{i=0}^{2n} W_i^{(c)} \left(\zeta_k^{(i)} - \hat{\mathbf{z}}_k \right) \left(\zeta_k^{(i)} - \hat{\mathbf{z}}_k \right)^\top + \mathbf{R}. \quad (22)$$

The cross-covariance between state and measurement is:

$$\mathbf{P}_{xz} = \sum_{i=0}^{2n} W_i^{(c)} \left(\chi_{k|k-1}^{(i)} - \hat{\mathbf{x}}_{k|k-1} \right) \left(\zeta_k^{(i)} - \hat{\mathbf{z}}_k \right)^\top. \quad (23)$$

d) *Update Step*: The Kalman gain and posterior update are:

$$\mathbf{K}_k = \mathbf{P}_{xz} \mathbf{S}_k^{-1}, \quad (24)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \hat{\mathbf{z}}_k), \quad (25)$$

$$\mathbf{P}_{k|k} = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top. \quad (26)$$

3) *Strengths and Weaknesses*:

a) *Strengths*:

- Handles strong nonlinearities better than the EKF.
- Does not require analytic Jacobians, which is beneficial for complex nonlinear and nonholonomic vehicle models.
- Typically more accurate and stable than the EKF for the same model and noise assumptions.

b) *Weaknesses*:

- Propagating $2n + 1$ sigma points instead of a single mean plus Jacobian introduces higher computational cost.
- Requires tuning of parameters (α , β , and κ) in addition to $\mathbf{Q}$ and $\mathbf{R}$.
- Still assumes Gaussian noise and unimodal distributions.
- Can be numerically unstable if parameters are poorly chosen or if the covariance loses positive definiteness.

4) *Computational Cost*:

- $2n + 1$ evaluations of the dynamics and measurement models per step.
- Matrix operations remain $\mathcal{O}(n^3)$, but with a larger constant factor than the EKF.

5) *Suitability for Small Autonomous Vehicles*: The UKF is attractive when:

- Dynamics and measurements are strongly nonlinear (e.g., 3D attitude, range/bearing sensors).
- Derivation of Jacobians is difficult or undesirable.

On a small AUV onboard computer, the UKF is often lightweight enough if:

- The state dimension is kept minimal.
- The update rate is moderate (~ 10 – 50 Hz).

6) *Open-Source Availability*:

- **ROS** `robot_localization`: `ekf_localization_node` and `ukf_localization_node` with the same interface.
- **FilterPy**: provides a tested UKF implementation in Python.
- **MATLAB / Simulink**: built-in UKF objects and blocks for online state estimation.

C. *Particle Filters (PF)*

1) *Conceptual Description*: Particle filters represent the belief over the state as a set of weighted samples (particles). They implement the general Bayesian filtering recursion directly:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}) \propto p(\mathbf{z}_k | \mathbf{x}_k) \int p(\mathbf{x}_k | \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1}. \quad (27)$$

The integral is approximated by particles:

- Propagate each particle through the dynamics with sampled process noise.
- Weight particles by measurement likelihood.
- Remove low-weight particles and replicate high-weight ones via resampling.

This allows particle filters to handle nonlinear, non-Gaussian, and multi-modal distributions, directly addressing the core limitation of EKF and UKF [7].

2) *Mathematical Formulation*: Consider the nonlinear system

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad (28)$$

with process and measurement noise $\mathbf{w}_k, \mathbf{v}_k$.

The posterior is approximated as

$$p(x_k | z_{1:k}) \approx \sum_{i=1}^N w_k^{(i)} \delta(x_k - x_k^{(i)}), \quad (29)$$

where $x_k^{(i)}$ is the i -th particle, $w_k^{(i)}$ its weight, and $\delta(\cdot)$ the Dirac delta.

Prediction:

$$x_k^{(i)} \sim p(x_k | x_{k-1}^{(i)}, u_{k-1}). \quad (30)$$

Update:

$$w_k^{(i)} \propto w_{k-1}^{(i)} p(z_k | x_k^{(i)}), \quad (31)$$

followed by normalization. When the effective sample size is low, a resampling step is performed. The state estimate can be obtained as the weighted mean of the particles.

3) Strengths and Weaknesses:

a) Strengths:

- Handles severe nonlinearities and non-Gaussian noise naturally.
- Can represent multi-modal posteriors.
- It is easy to incorporate complex sensor models and constraints.

b) Weaknesses:

- Computational cost scales linearly with the number of particles N , which must be large for higher-dimensional states.
- The number of required particles grows rapidly with state dimension.
- Requires careful proposal design and resampling strategies to avoid degeneracy.

4) Computational Cost: Per time step:

- N evaluations of the dynamics model,
- N evaluations of the measurement likelihood,
- Resampling with complexity $\mathcal{O}(N)$.

5) *Suitability for Small Autonomous Vehicles:* Particle filters are particularly attractive when:

- The state dimension is low (e.g., 2D or 3D pose),
- Sensors induce highly non-Gaussian or multi-modal beliefs (e.g., sonar localization in clutter).

For small AUVs, particle filters are realistic for low-dimensional localization (e.g., pose plus heading) given a map, such as Monte Carlo localization with bathymetric or feature-based maps. PFs are typically not used for full 6-DOF dynamic state estimation onboard very small vehicles due to computational burden; EKF and UKF are more common in this case.

6) Open-Source Availability:

- **ROS amcl:** Adaptive Monte Carlo Localization (particle filter) for 2D mobile robot localization.
- **FilterPy:** includes generic particle filter implementations in Python.

D. Comparison and Discussion

The three filters discussed above represent distinct trade-offs between estimation accuracy, computational complexity, and robustness to nonlinearity and uncertainty. The EKF offers the lowest computational cost and remains a reliable baseline for small autonomous vehicles when system nonlinearities are moderate and accurate Jacobians are available; however, its reliance on first-order linearization can degrade performance as nonlinearity increases. The UKF improves estimation accuracy by propagating uncertainty through nonlinear dynamics using sigma points, providing greater robustness to nonlinear motion and measurement models at the cost of increased computation and parameter tuning. Particle Filters offer the highest representational flexibility by capturing non-Gaussian and multi-modal state distributions, but their computational cost scales with the number of particles, making them impractical for high-dimensional full-state estimation on resource-constrained platforms.

TABLE I
QUALITATIVE COMPARISON OF STATE ESTIMATORS FOR SMALL AUVs.

Algorithm	Nonlinearity	Accuracy	Comp. Load
EKF	Low-Medium	Medium	Low
UKF	Medium-High	High	Medium
PF	High (non-Gaussian)	High	High

1) Estimator Accuracy vs. Computational Load:

E. Role of State Estimation in the Navigation Stack

State estimation provides the interface between sensor measurements and downstream navigation modules. Planning and control algorithms do not operate directly on raw sensor data due to noise, bias, and asynchrony. Instead, they assume access to a state estimate $\hat{x}_k$ produced by an estimator such as an EKF, UKF, or PF, together with associated covariance or quality indicators. In marine systems, state estimation is therefore a prerequisite for reliable path planning and safety-critical control.

III. PATH PLANNING ALGORITHMS

Path planning is responsible for generating collision-free routes that move an autonomous vehicle from an initial state to a goal while respecting environmental, kinematic, and dynamic constraints. This section reviews both classical graph-search methods (A*, D*, Hybrid A*) and sampling-based algorithms (RRT*, PRM, BIT*, FMT*), as well as continuous trajectory optimization approaches, with an emphasis on their properties in cluttered, GPS-denied marine environments.

A. A* Search Algorithm

1) *Conceptual Description:* The A* algorithm is a graph-search method that computes the lowest-cost path between a start node and a goal node [8]. For each node n , A* maintains the cost $g(n)$ from the start and a heuristic estimate $h(n)$ of the remaining cost to the goal. The evaluation function

$$f(n) = g(n) + h(n) \quad (32)$$

ranks nodes according to their estimated total path cost. At each iteration, A* expands the open node with the smallest $f(n)$, updates its neighbors, and repeats until a goal node is reached or no nodes remain to explore.

Algorithm 1 A* Search Algorithm

Require: Graph $G = (V, E)$, start node s , goal set $\mathcal{G}$, edge cost $c(u, v) \geq 0$, heuristic $h(\cdot)$

Ensure: A minimum-cost path from s to some $g \in \mathcal{G}$, if one exists

```

1: OPEN  $\leftarrow \{s\}$  {frontier of nodes to explore}
2: CLOSED  $\leftarrow \emptyset$ 
3:  $g(s) \leftarrow 0$ 
4:  $f(s) \leftarrow g(s) + h(s)$ 
5:  $\text{parent}(s) \leftarrow \text{NIL}$ 
6: while OPEN  $\neq \emptyset$  do
7:   Select  $n \in \text{OPEN}$  with minimal  $f(n)$ 
8:   if  $n \in \mathcal{G}$  then
9:     return path obtained by following  $\text{parent}(\cdot)$  from  $n$ 
       back to  $s$ 
10:  end if
11:  Move  $n$  from OPEN to CLOSED
12:  for all neighbors  $n'$  of  $n$  do
13:     $g_{\text{tent}} \leftarrow g(n) + c(n, n')$ 
14:    if  $n' \notin \text{OPEN}$  and  $n' \notin \text{CLOSED}$  then
15:       $\text{parent}(n') \leftarrow n$ 
16:       $g(n') \leftarrow g_{\text{tent}}$ 
17:       $f(n') \leftarrow g(n') + h(n')$ 
18:      Add  $n'$  to OPEN
19:    else if  $g_{\text{tent}} < g(n')$  then
20:       $\text{parent}(n') \leftarrow n$ 
21:       $g(n') \leftarrow g_{\text{tent}}$ 
22:       $f(n') \leftarrow g(n') + h(n')$ 
23:    if  $n' \in \text{CLOSED}$  then
24:      Move  $n'$  from CLOSED to OPEN
25:    end if
26:  end if
27: end for
28: end while
29: return failure {no path from  $s$  to any  $g \in \mathcal{G}$ }
```

2) *Algorithm Outline:*

3) *Completeness and Optimality:* A* is complete on finite graphs with nonnegative edge costs: if a path from s to g exists, the algorithm will eventually find one. Optimality depends on the heuristic $h(n)$. If $h(n)$ is *admissible* and *consistent* (satisfies a triangle inequality along edges), then A* is guaranteed to return a path with minimum cost [9].

4) *Computational Aspects and Suitability for Marine Navigation:* The worst-case time and memory complexity of A* can be high, as the number of expanded nodes may grow exponentially with problem size. However, with a well-chosen heuristic, A* often explores only a subset of the graph and is efficient for 2D or low-resolution 3D grids. In marine applications, A* is widely used on occupancy or bathymetric grids to generate collision-free reference paths for AUVs and

USVs. The resulting piecewise-linear paths typically require post-processing (e.g., smoothing or conversion to waypoints compatible with vehicle kinematics).

*B. Hybrid A**

1) *Conceptual Description:* Hybrid A* extends classical A* to systems with nonholonomic constraints and limited turning radius [10]. Instead of planning purely on a discrete grid in position space, hybrid A* searches in a continuous state space while still using an A*-style best-first search with cost

$$f(n) = g(n) + h(n), \quad (33)$$

where $g(n)$ is the cumulative cost and $h(n)$ a heuristic to the goal.

Successors are generated by forward-simulating vehicle dynamics or motion primitives from the current state, rather than by simple grid moves. States are discretized for bookkeeping (e.g., (x, y, ψ) cells), but each node retains a continuous pose and is guaranteed to be kinematically feasible. Collision checking is performed along motion segments.

2) *Key Properties and Suitability:*

a) *Completeness and Optimality:* Hybrid A* is not globally optimal in a strict continuous sense, but under a fixed discretization and motion primitive set it is:

- *Resolution complete:* if a solution exists within the chosen discretization, it will be found.
- *Resolution optimal:* it returns the minimal-cost trajectory among the discrete set represented by the search.

b) *Runtime:* Hybrid A* is more computationally expensive than grid-based A* because:

- Each node expansion requires forward simulation,
- Collision checking is performed along continuous trajectories,
- The branching factor is increased by multiple control options.

In practice, hybrid A* is often used with:

- Coarser grids than pure A* to control the number of nodes.
- Strong heuristics to focus the search.
- Limits on search depth or time, yielding anytime or bounded-suboptimal behavior when needed.

c) *Smoothness and Kinematic Feasibility:* Hybrid A* directly produces kinematically feasible paths because all expansions integrate the vehicle's motion model. However:

- The resulting path is typically only C^0 continuous in curvature.
- A post-processing step is often applied (e.g., gradient-based optimization or MPC-based refinement) to improve smoothness, remove small zig-zags, and better respect dynamic constraints.

For marine vehicles, this is attractive because it avoids infeasible sharp turns that would appear in purely geometric A* or D* paths.

d) *Scalability*: The state dimension (e.g., (x, y, z, ψ)) and fine discretization can make hybrid A* expensive in large maps:

- The search space grows with the number of spatial cells times the number of discretized headings (and possibly depth).
- The branching factor grows with the number of motion primitives.

Hybrid A* scales reasonably for 2D or 2.5D planning, but becomes challenging for full 3D planning over large volumes. Hierarchical approaches are commonly used to improve scalability.

e) *Use in GPS-Denied / Cluttered Environments*: In GPS-denied underwater environments, hybrid A* is typically applied on a local or global occupancy grid obtained from sonar, acoustic imaging, or other onboard sensors:

- **Local planning**: Using a map built from onboard sensors and a state estimate from EKF/UKF/PF, hybrid A* can generate kinematically feasible avoidance maneuvers around detected obstacles.
- **Global planning**: When a coarse prior map is available, hybrid A* can compute a route that respects turning radius and dynamic limits, making it suitable for narrow channels, harbor entries, and cluttered seabed operations.

Its main advantage over purely geometric planners (A*, D*, PRM) in these settings is the ability to incorporate vehicle motion constraints directly, reducing the load on the downstream controller working together with its constraints.

In the context of this work, hybrid A* provides a bridge between discrete planning on occupancy maps and continuous, kinematically feasible trajectories. It is especially relevant when downstream controllers (e.g., CLF-CBF or MPC-based) are expected to track paths that already respect turning-radius and collision constraints, reducing corrective control effort in cluttered underwater environments. However, a map (prior or online) is still required; in completely unknown environments hybrid A* will struggle in the same way as grid A*, so some form of underwater mapping is necessary to deploy this class of planners.

3) *Algorithm Overview*: A high-level hybrid A* procedure:

- 1) Discretize (x, y) and heading ψ .
- 2) Initialize the open set with the start state.
- 3) At each step, select the node with minimal $f(n)$.
- 4) Generate successors by applying motion primitives, simulate, and collision-check.
- 5) Insert valid successors, update costs and parents.
- 6) Terminate when a node is within tolerance of the goal.

C. *Dynamic Replanning in Partially Known Environments*: D*

1) *Conceptual Description*: The D* algorithm extends A* to unknown or changing environments by incrementally repairing shortest paths when edge costs change, rather than replanning from scratch [11]. The environment is modeled as a graph; D* maintains, for each state, a cost-to-go estimate and a backpointer to its successor on the current optimal path.

The robot follows backpointers to the goal while D* updates locally when obstacles or costs change.

2) *Completeness and Optimality*: For a static environment with a fixed cost map and consistent heuristic, D* computes the same optimal paths as A* and is complete and optimal on finite graphs. In dynamic or partially known environments, D* preserves optimality with respect to the latest cost information by updating affected states and predecessors.

3) *Runtime*: D* is more efficient than repeated A* replanning in environments where only local map changes occur. It maintains an *OPEN* list of states to update and focuses computation on cost changes (RAISE/LOWER updates), reducing redundant work.

4) *Path Smoothness and Scalability*: Paths produced by D* are discrete and grid-aligned, similar to A*. They often require smoothing and conversion to dynamically feasible trajectories. D* inherits the scalability limitations of grid-based search, but reuses previous computation, which is advantageous in large, slowly changing maps.

5) *Use in GPS-Denied and Cluttered Environments*: In GPS-denied marine environments, maps are often incomplete and obstacles detected at short range. D* is well suited for such settings: an initial path is planned on a coarse map, then incrementally repaired as new information is obtained via sonar or other sensors.

6) *Simplified Algorithmic Outline*: A simplified D* replanning loop:

Algorithm 2 Simplified D* Replanning Loop

Require: Graph $G = (V, E)$, start state s , goal state g , edge cost $c(u, v)$

Ensure: Incrementally updated minimum-cost path from s to

```

 $g$ 
1: Initialize cost-to-go  $J(v) \leftarrow \infty$  and  $\text{parent}(v) \leftarrow \text{NIL}$ 
   for all  $v$ 
2:  $J(g) \leftarrow 0$ ;  $\text{OPEN} \leftarrow \{g\}$ 
3: function COMPUTESHORTESTPATHS():
4:   while  $\text{OPEN} \neq \emptyset$  do
5:     Remove state  $n$  with smallest key from  $\text{OPEN}$ 
6:     for all predecessors  $p$  of  $n$  do
7:        $J_{\text{tent}} \leftarrow c(p, n) + J(n)$ 
8:       if  $J_{\text{tent}} < J(p)$  then
9:          $J(p) \leftarrow J_{\text{tent}}$ 
10:         $\text{parent}(p) \leftarrow n$ 
11:        Insert or update  $p$  in  $\text{OPEN}$ 
12:       end if
13:     end for
14:   end while
15: end function

```

During execution, when edge costs change (e.g., new obstacles), affected states are reinserted into *OPEN* and *COMPUTESHORTESTPATHS* is called again.

D. *Sampling-Based Planning*: RRT and RRT*

Rapidly-exploring Random Trees (RRT) are sampling-based motion planning algorithms that explore high-dimensional

spaces by incrementally growing a tree rooted at the start configuration [12]. The basic RRT algorithm is probabilistically complete but not optimal. RRT* augments RRT with best-parent selection and local rewiring to achieve asymptotic optimality [13].

a) Algorithm Outline (RRT / RRT):*

- 1) Initialize tree $\mathcal{T}$ with x_{start} .
- 2) **Loop:**
 - a) Sample random state x_{rand} .
 - b) Find nearest node x_{near} (RRT) or neighborhood $\mathcal{N}_{\text{near}}$ (RRT*).
 - c) Steer toward x_{rand} with step size Δq to obtain x_{new} .
 - d) If the edge to x_{new} is collision-free, add x_{new} to $\mathcal{T}$:
 - RRT: parent is x_{near} .
 - RRT*: parent is the neighbor that minimizes cost-to-come plus edge cost.
 - e) RRT*: rewire neighbors if routing through x_{new} lowers their cost.

b) Comparison and Properties: In terms of completeness, optimality, runtime, smoothness, scalability, and operation in GPS-denied environments, RRT and RRT* have complementary characteristics. Both algorithms are *probabilistically complete*: if a feasible path exists, the probability that they find one approaches 1 as the number of samples increases. Standard RRT is not optimal and often returns jagged, suboptimal, piecewise-linear paths that require post-processing or smoothing, whereas RRT* is asymptotically optimal due to its local rewiring step. This comes at the cost of higher per-iteration computation, since each new node must search a neighborhood and potentially update multiple parents.

From a runtime and scalability perspective, both RRT and RRT* scale better to high-dimensional state spaces than grid-based planners like A*, because they avoid explicit discretization of the configuration space. RRT is faster per sample and often preferred when a “good enough” path is sufficient or when real-time constraints are tight. RRT* is more suitable when plan quality (e.g., path length, energy, or time) is important and some additional computation is acceptable.

For small autonomous marine vehicles operating in cluttered, GPS-denied environments, sampling-based planners are attractive because they can incorporate complex vehicle constraints and nonconvex obstacles directly through collision checking on continuous trajectories. In practice, RRT or RRT* is often combined with a separate trajectory-smoothing or path-following controller, which is then integrated with the state estimation and CBF-based safety layer discussed in later sections.

E. Probabilistic Roadmaps (PRM)

Probabilistic Roadmaps (PRM) are sampling-based, *multi-query* planners designed for high-dimensional configuration spaces [14]. PRM builds a roadmap graph approximating the connectivity of the free space and then answers individual

queries by connecting start and goal configurations to the roadmap and running graph search.

a) Roadmap Construction:

- 1) Sample N random configurations; retain those that are collision-free.
- 2) For each sample, connect to k nearest neighbors (or within radius r) via a local planner if collision-free.

b) Query Phase:

- 1) Insert start and goal into the roadmap.
- 2) Connect them to nearby nodes.
- 3) Use Dijkstra or A* to find a path on the roadmap.

PRM is probabilistically complete; optimal variants (PRM*) are asymptotically optimal. In marine applications with static maps (e.g., bathymetric charts), PRM is well suited when many different start–goal queries must be answered.

1) Properties:

a) Completeness and Optimality: PRM is *probabilistically complete*: if a feasible path exists, the probability that PRM finds it approaches 1 as the number of samples $N \rightarrow \infty$. In its basic form it is not guaranteed to find the globally optimal path in terms of length or cost. Optimal variants such as PRM* can asymptotically converge to the optimal solution, but at increased computational cost; in many marine applications, the basic PRM is used as a practical trade-off between solution quality and runtime.

b) Runtime: The roadmap construction phase can be computationally expensive due to repeated sampling, nearest-neighbor searches, and collision checks, especially in cluttered 3D environments. However, once the roadmap is built, individual planning queries are very fast: they reduce to connecting two nodes and performing a graph search on a relatively sparse graph. This makes PRM attractive when many start–goal queries must be solved in the same static environment (multi-query setting).

c) Smoothness: The raw PRM path is a polyline sequence of straight segments between sampled configurations. It is typically not dynamically feasible or smooth for an AUV. In practice, PRM is often combined with:

- Path smoothing (shortcutting, spline fitting), and/or
- A downstream path-following or trajectory-tracking controller that enforces vehicle dynamics and curvature limits.

Thus PRM provides a *topologically valid* route, while smoothness is handled in a post-processing or control layer.

d) Scalability: PRM scales better than grid-based planners (e.g., A* on an occupancy grid) for high-dimensional configuration spaces, because it avoids exhaustive discretization. However, the number of samples required still grows quickly with dimension, and collision checking can dominate runtime. Use of efficient nearest-neighbor data structures and lazy collision checking can significantly improve scalability. For AUVs with a moderate-dimensional state (position + heading + depth), PRM is often practical, especially when the environment is static and the same roadmap can be reused.

e) *Use in GPS-Denied / Cluttered Environments:* For underwater vehicles operating with limited absolute positioning (GPS-denied) but access to prior maps, PRM can be used to precompute a roadmap *offline* in the map frame. Online, the AUV uses its state estimator (e.g., EKF/UKF) to localize within that map and plan on the roadmap. When the map is not available (pure exploration missions), building a high-quality PRM online is less convenient. In heavily cluttered environments (caves, under-ice channels, rocky seabeds), PRM can capture complex connectivity of free space better than simple potential-field methods, but it is less suited to rapidly changing environments (moving obstacles, time-varying no-go zones) unless the roadmap is periodically updated or augmented with local replanning around dynamic obstacles.

F. Batch Informed Trees (BIT*)

Batch Informed Trees (BIT*) unify heuristic graph search with sampling-based planning by searching an implicit random geometric graph in a best-first manner [15]. Samples are generated in batches; edges are ordered by a lower bound on solution cost (analogous to $f = g + h$) and processed in that order. BIT* is probabilistically complete and asymptotically optimal and often converges faster to high-quality solutions than RRT* and PRM* in moderately high dimensions.

BIT* restricts sampling to an informed subset of the state space once an initial solution is found, focusing computation on states that can improve the current solution.

1) *Algorithm Outline:* A high-level outline of BIT* is as follows:

1) Initialization:

- a) Initialize the tree T with the start state x_s as the root.
- b) Set the best solution cost $c_{\text{best}} \leftarrow \infty$.
- c) Initialize the set of free samples $X_{\text{samples}} \leftarrow \emptyset$.

2) Main loop (run until time or iteration limit):

- a) If there are no more candidate edges to process, generate a new batch of samples:
 - i) If $c_{\text{best}} = \infty$, draw N samples uniformly from X_{free} .
 - ii) Otherwise, draw N samples from the *informed set*, i.e., the subset of X_{free} that can still yield a path with cost $< c_{\text{best}}$.
- iii) Add these samples to X_{samples} .
- iv) For each tree vertex v and each nearby sample $x \in X_{\text{samples}}$ within a connection radius r_n , create a candidate edge (v, x) with an associated lower bound on the total path cost:

$$f_{\text{edge}}(v, x) = g_T(v) + \hat{c}(v, x) + \hat{h}(x),$$

where $g_T(v)$ is the current cost-to-come in the tree, $\hat{c}(v, x)$ is a lower bound on the cost from v to x (e.g., Euclidean distance), and $\hat{h}(x)$ is a heuristic lower bound from x to the goal.

- v) Insert all such candidate edges into a priority queue ordered by f_{edge} .

b) While the priority queue is not empty:

- i) Pop the candidate edge (v, x) with the smallest $f_{\text{edge}}(v, x)$.
- ii) If $f_{\text{edge}}(v, x) \geq c_{\text{best}}$, discard the edge (it cannot improve the best solution) and continue.
- iii) If the new cost-to-come

$$g_{\text{new}} = g_T(v) + c(v, x)$$

is smaller than the current $g_T(x)$ (or if x is not yet in the tree), and the local connection (v, x) is collision-free, then:

- A) Add x to the tree (or rewire its parent to v).
- B) Update $g_T(x) \leftarrow g_{\text{new}}$.
- C) If x is in the goal region, update $c_{\text{best}} \leftarrow \min\{c_{\text{best}}, g_T(x)\}$.

c) Optionally, prune vertices and samples that cannot possibly be part of any solution with cost $< c_{\text{best}}$ to limit memory and focus computation.

The algorithm returns the best path found in the tree from x_s to the goal when terminated.

2) Discussion:

a) *Completeness and optimality:* BIT* is *probabilistically complete*: the probability of finding a solution, if one exists, approaches one as the number of samples grows. Moreover, under standard assumptions on the connection radius and heuristics, BIT* is *asymptotically optimal*: the cost of the best solution converges almost surely to the global optimum as the number of samples tends to infinity. These guarantees are analogous to those of RRT* and PRM*, but BIT* adds heuristic-guided search and batch processing.

b) *Runtime:* Runtime is dominated by three operations: sampling, maintaining the priority queue of candidate edges, and collision checking. For a batch of N samples, building and processing the candidate edge queue typically scales as $\mathcal{O}(N \log N)$ plus the cost of local collision checks. In practice, BIT* tends to find high-quality solutions significantly faster than RRT* or PRM* in moderately high dimensions due to its A*-like ordering of candidate edges and informed sampling, even though each iteration is more expensive than a single RRT* extension.

c) *Smoothness:* BIT* optimizes with respect to a given cost metric (e.g., Euclidean path length), so the resulting paths are typically shorter and less jagged than those from basic RRT. However, the output is still a piecewise-linear sequence of waypoints, and additional trajectory smoothing or time-parameterization is usually applied before sending the path to a dynamic controller, especially for vehicles with nonholonomic or dynamic constraints.

d) *Scalability:* BIT* scales better than grid-based planners in high-dimensional configuration spaces because it does not require discretizing the space. Compared to other optimal sampling-based planners, it exhibits improved convergence to high-quality solutions in moderately high dimensions, thanks to its heuristic-guided search and batch processing. Memory usage grows with the number of stored samples and tree

vertices, so batch size and pruning strategies must be chosen carefully.

e) *Use in GPS-denied / cluttered environments.*: For small marine vehicles operating in GPS-denied, cluttered environments, BIT* is attractive as an offline or replanning path planner. It only requires a configuration-space representation and a collision checker, and it can incorporate cost terms that reflect path length, clearance from obstacles, or energy usage. The informed sampling region helps concentrate computation in corridors that can actually improve the path, which is beneficial in narrow underwater channels or around obstacles. As with most static planners, extensions or frequent replanning are required to handle strong time-varying ocean currents or moving obstacles.

G. Fast Marching Tree (FMT*)

The Fast Marching Tree (FMT*) algorithm is a batch, sampling-based planner that grows a tree from a set of samples using a dynamic-programming recursion similar to Dijkstra's algorithm [16]. FMT* constructs an implicit random geometric graph and connects nodes in order of increasing cost-to-come, performing collision checking lazily and yielding asymptotically optimal solutions.

FMT* is particularly efficient when collision checking is expensive and is attractive for offline or global planning in static marine environments.

1) *Algorithm Outline*: Let V be the set of samples in free space and let $c(\cdot, \cdot)$ denote a metric (e.g., Euclidean distance) used as edge cost.

1) Sampling and initialization

- a) Draw n samples in $\mathcal{X}_{\text{free}}$ and form the set V .
- b) Set $V_{\text{open}} = \{x_{\text{init}}\}$, $V_{\text{closed}} = \emptyset$, and $V_{\text{unvisited}} = V \setminus \{x_{\text{init}}\}$.
- c) Initialize cost-to-come $J(x_{\text{init}}) = 0$.

2) Main loop (repeat while V_{open} is non-empty and no goal has been connected):

- a) Select $z \in V_{\text{open}}$ with minimal $J(z)$.
- b) Compute the set of unvisited neighbors within radius r_n :

$$N_z = \{y \in V_{\text{unvisited}} : \|y - z\| \leq r_n\}.$$

c) For each $y \in N_z$:

- i) Find the set of potential parents

$$X_y = \{x \in V_{\text{open}} \cup V_{\text{closed}} : \|y - x\| \leq r_n\}.$$

- ii) Choose a parent that minimizes cost-to-come

$$x^* = \arg \min_{x \in X_y} (J(x) + c(x, y)).$$

- iii) If the edge (x^*, y) is collision-free, add y to the tree with parent x^* and set $J(y) = J(x^*) + c(x^*, y)$.

- iv) Mark y for insertion into V_{open} and remove it from $V_{\text{unvisited}}$.

- d) Move z from V_{open} to V_{closed} .

- e) Insert all newly connected nodes into V_{open} .

3) Termination

- a) If some node $x \in V_{\text{open}} \cap \mathcal{X}_{\text{goal}}$ is reached, reconstruct the path from x_{init} to x via parent pointers.
- b) If V_{open} becomes empty with no goal reached, report failure for the given sample set.

2) Properties and Discussion:

a) *Completeness and optimality.*: FMT* is probabilistically complete and asymptotically optimal under standard assumptions on the connection radius (e.g., $r_n \propto (\log n/n)^{1/d}$ for state-space dimension d). As the number of samples $n \rightarrow \infty$, the cost of the returned path converges in probability to the optimal cost. Compared to PRM* and RRT*, it often exhibits faster convergence to high-quality solutions in high-dimensional spaces and when collision checking is expensive [16].

b) *Runtime and collision checking.*: The dominant costs are nearest-neighbor queries and a relatively small number of collision checks. Because FMT* performs “lazy” collision checking (one edge per candidate neighbor) and expands nodes in order of increasing cost, it tends to use collision checking more efficiently than PRM* or RRT* for a fixed time budget.

c) *Path quality and smoothness.*: As with other sampling-based planners, FMT* produces piecewise-geodesic paths (e.g., straight segments in configuration space). The resulting path is typically of high quality in terms of cost, but may require post-processing (shortcutting, smoothing) before being used as a reference for low-level controllers on marine vehicles.

d) *Scalability.*: FMT* scales well to higher-dimensional problems due to its batch structure and ordered exploration of the cost-to-come landscape. However, like other batch planners, memory and nearest-neighbor overhead grow with the number of samples; careful implementation of neighbor search is important for large-scale problems.

e) *Use in GPS-denied / cluttered marine environments.*: For small AUVs in cluttered, map-based environments (e.g., under-ice cavities), FMT* is well-suited as an offline or replanning global planner: a set of samples can be drawn in a known occupancy map, and FMT* can generate a near-optimal energy- or distance-minimizing path between waypoints. The resulting path then serves as a reference trajectory for local guidance and control (e.g., LOS guidance, MPC or CBF-based controllers), while the batch nature of FMT* remains compatible with limited onboard compute.

H. Trajectory Optimization Approaches

Trajectory optimization treats planning as a continuous optimal control problem, directly computing state-control trajectories that minimize a cost under dynamics and constraints:

$$\min_{x(\cdot), u(\cdot)} J = \int_0^T \ell(x(t), u(t)) dt + \phi(x(T)) \quad (34)$$

$$\text{s.t. } \dot{x}(t) = f(x(t), u(t)), \quad x(0) = x_0, \quad (35)$$

$$g(x(t), u(t)) \leq 0, \quad h(x(T)) = 0. \quad (36)$$

TABLE II
QUALITATIVE COMPARISON OF PATH PLANNING ALGORITHMS IN TERMS
OF OPTIMALITY AND RUNTIME.

Planner	Optimality	Runtime	Remarks
A*	Opt. (grid, adm. heur.)	High (large grids)	Deterministic search, single query
D*	Opt. (grid)	Lower than repeated A*	Incremental replanning on changing maps
RRT	No global optimality	Low per iteration	Fast feasibility, rough paths
RRT*	Asymptotically opt.	Higher than RRT	Cost converges as samples grow
PRM	No (basic); PRM* opt.	High offline, low online	Good for multi-query environments
FMT*	Asymptotically opt.	Efficient in batches	Good when collision checking is expensive
BIT*	Asymptotically opt.	Anytime (improves over time)	Heuristic-guided sampling and search
Hybrid A*	Resolution-opt.	Higher than A*	Handles kinematics, continuous states
Traj. opt.	Local optimum	Solver-dependent	Handles dynamics and hard constraints

In practice, (34)–(36) is discretized and solved as a non-linear program using direct shooting or collocation. In marine robotics, trajectory optimization is often implemented as (non-linear) model predictive control (MPC/NMPC), which solves such problems at each time step over a receding horizon [17].

Trajectory optimization provides:

- Direct handling of nonlinear dynamics,
- Explicit constraint handling (inputs, states, obstacles),
- Smooth, dynamically feasible trajectories.

However, it is typically only locally optimal and computationally expensive, making it best suited as a local refinement layer on top of graph/sampling-based planners.

I. Planner Optimality vs. Runtime

IV. CONTROL STRATEGIES FOR PATH TRACKING AND SAFETY

Path planning algorithms produce geometric paths or reference trajectories. To execute these on a physical AUV, a feedback controller must track the reference while rejecting disturbances and respecting constraints. This section reviews PID, LQR, MPC, and CBF-based safety-critical control with respect to stability, path-tracking capability, real-time feasibility, and integration with planners.

A. PID Control

Proportional–Integral–Derivative (PID) control remains the most widely deployed feedback method in marine systems due to its simplicity and low computational cost. For a scalar tracking error $e(t)$, a PID controller is

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \dot{e}(t), \quad (37)$$

with gains K_P, K_I, K_D .

a) Stability Guarantees: For linear SISO plants, classic frequency-domain tools (root locus, Bode plots) can tune PID gains to yield acceptable margins. For nonlinear, coupled AUV dynamics, stability guarantees are limited and rely heavily on empirical tuning.

b) Suitability for Path Tracking: PID is effective for setpoint regulation (e.g., depth, heading, speed) or slowly varying references. For full 3D path-following, PID is typically embedded in a guidance–control cascade (e.g., line-of-sight guidance feeding depth/heading commands).

c) Real-Time Feasibility: PID is trivial to implement on low-power microcontrollers and is suitable for very small platforms.

d) Integration with Planners: Planners provide waypoints or path segments; a guidance law converts these into reference commands tracked by PID loops. Constraint handling is implicit and limited.

B. Linear Quadratic Regulator (LQR)

The Linear Quadratic Regulator (LQR) provides an optimal state-feedback law for linear systems

$$\dot{x} = Ax + Bu, \quad (38)$$

minimizing

$$J = \int_0^\infty (x^\top Qx + u^\top Ru) dt, \quad (39)$$

with $Q \succeq 0$, $R \succ 0$. The optimal control is $u = -Kx$, with K from the algebraic Riccati equation [18].

a) Stability Guarantees: If (A, B) is stabilizable and $(Q^{1/2}, A)$ is detectable, LQR yields an asymptotically stable closed loop.

b) Suitability for Path Tracking: LQR is effective for tracking when deviations are small and a linearization around an operating point or nominal trajectory is valid. It does not explicitly handle hard constraints.

c) Real-Time Feasibility: Online evaluation reduces to matrix–vector multiplication and is lightweight.

d) Integration with Planners: Planners provide nominal trajectories; LQR is designed on error dynamics $x_e = x - x_{\text{ref}}$ to stabilize around them.

C. Model Predictive Control (MPC)

Model Predictive Control (MPC) optimizes future control actions over a finite horizon, enforcing constraints:

$$\min_{\{u_{k|k}, \dots, u_{k+N-1|k}\}} \sum_{j=0}^{N-1} \ell(x_{k+j|k}, u_{k+j|k}) + \ell_f(x_{k+N|k}) \quad (40)$$

$$\text{s.t. } x_{k+j+1|k} = f(x_{k+j|k}, u_{k+j|k}), \quad (41)$$

$$x_{k+j|k} \in \mathcal{X}, \quad u_{k+j|k} \in \mathcal{U}, \quad (42)$$

with only $u_{k|k}^*$ applied, and the horizon receding at each step.

a) Stability Guarantees: For linear MPC with appropriate terminal ingredients, closed-loop stability can be guaranteed. Nonlinear MPC stability is more involved but can be established under certain conditions.

b) *Suitability for Path Tracking*: MPC is well suited when dynamics, constraints, and multi-objective trade-offs are significant, such as:

- Limited thrust, rate limits, and attitude bounds,
- Aggressive or time-varying trajectories,
- Obstacle and safety constraints.

c) *Real-Time Feasibility*: The main limitation is solving an optimization problem at each step. Linear MPC is feasible on embedded hardware for small state dimensions and short horizons; nonlinear MPC often requires more capable processors or reduced update rates.

d) *Integration with Planners*: Planners provide reference trajectories; MPC computes control inputs that track these references while respecting constraints. Obstacle constraints and safety margins can be integrated into the MPC formulation.

D. CBF-Based Safety-Critical Control

Control Barrier Functions (CBFs) provide a Lyapunov-like framework for enforcing safety as forward invariance of a set [19]. Let

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}, \quad (43)$$

with a differentiable $h : \mathbb{R}^n \rightarrow \mathbb{R}$. For control-affine systems $\dot{x} = f(x) + g(x)u$, a CBF condition is

$$L_f h(x) + L_g h(x)u \geq -\alpha(h(x)), \quad (44)$$

with α an extended class- $\mathcal{K}$ function.

At each step, a QP computes u close to a nominal input u_{des} while satisfying all CBF constraints:

$$\min_u \|u - u_{\text{des}}\|^2 \quad (45)$$

$$\text{s.t. } L_f h_i(x) + L_g h_i(x)u \geq -\alpha_i(h_i(x)), \forall i. \quad (46)$$

a) *Stability Guarantees*: CBFs guarantee forward invariance of $\mathcal{C}$, i.e., safety, but do not ensure convergence to goals. They are often combined with Control Lyapunov Functions (CLFs) in a CLF–CBF QP to blend performance and safety.

b) *Suitability for Path Tracking*: CBFs modify a nominal tracking controller only when necessary to prevent constraint violation. This makes them especially attractive for:

- Collision avoidance,
- Connectivity maintenance,
- Line-of-sight and depth constraints.

c) *Real-Time Feasibility*: For low-dimensional systems with few constraints, the resulting QP is small and can be solved in real time using embedded solvers.

d) *Integration with Planners*: Planner outputs (paths, connectivity requirements, LOS constraints) can be translated into barrier functions, while nominal controllers (PID, LQR, MPC) track references. The CBF layer enforces safety online in a minimally invasive way, enabling connectivity-preserving and collision-avoiding behavior for multi-AUV systems [20], [21].

TABLE III
QUALITATIVE COMPARISON OF CONTROLLER ROBUSTNESS VS.
IMPLEMENTATION COMPLEXITY FOR SMALL AUTONOMOUS MARINE
VEHICLES.

Controller	Robustness	Model dep.	Implementation complexity
PID	Low–Medium	Low	Low
LQR	Medium (local)	High (linear model)	Low–Medium
MPC	Medium–High	High (prediction model)	High
CBF + nominal	High for safety	Medium–High	Medium–High (online QP)

E. Controller Robustness vs. Complexity

REFERENCES

- [1] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] Y. Bar-Shalom, X.-R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY: John Wiley & Sons, 2001.
- [3] T. Moore and S. Nuss, “robot_localization: A multi-sensor EKF/UKF-based state estimation package,” ROS package, 2014, available at: http://wiki.ros.org/robot_localization.
- [4] R. R. Labbe, “Filterpy: Kalman filtering and optimal estimation in python,” Online book and software library, 2014, available at: <https://github.com/rlabbe/filterpy>.
- [5] *Sensor Fusion and Tracking Toolbox*, MATLAB Documentation, The MathWorks, Inc., Natick, MA, 2023, available at: <https://www.mathworks.com/help/fusion>.
- [6] S. J. Julier and J. K. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [7] A. Doucet, N. de Freitas, and N. Gordon, Eds., *Sequential Monte Carlo Methods in Practice*. New York, NY: Springer, 2001.
- [8] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [9] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [10] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, “Path planning for autonomous vehicles in unknown semi-structured environments,” *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010.
- [11] A. Stentz, “Optimal and efficient path planning for partially-known environments,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 1994, pp. 3310–3317.
- [12] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Department of Computer Science, Iowa State University, Tech. Rep. TR 98-11, 1998.
- [13] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [14] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [15] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.
- [16] L. Janson, E. Schmerling, A. A. Clark, and M. Pavone, “Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions,” *The International Journal of Robotics Research*, vol. 34, no. 7, pp. 883–921, 2015.
- [17] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI: Nob Hill Publishing, 2017.
- [18] F. L. Lewis, D. L. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed. Hoboken, NJ: John Wiley & Sons, 2012.

- [19] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proceedings of the 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [20] Y. Yang, C. Manzie, and Y. Pu, "A control barrier function composition approach for multi-agent systems in marine applications," *arXiv preprint arXiv:2403.14369*, 2024.
- [21] M. N. Nguyen, S. McIlvanna, J. Close, and M. Van, "Real-time reconfiguration and connectivity maintenance for AUVs network under external disturbances using distributed nonlinear model predictive control," *arXiv preprint arXiv:2403.15671*, 2024.